Contour Software — Java Assessment

OOP • Java • DSA • SQL • REST APIs • Git • Spring Boot • Interview Q&A;

Muhammad Faizan | Trainee Software Developer | iMocha Assessment Prep

1. OOP — OBJECT ORIENTED PROGRAMMING

Concept	Definition	Java Example
Class	Blueprint for creating objects	class Car { String brand; }
Object	Instance of a class	Car c = new Car();
Encapsulation	Hide data using private + getters/setters	private int age; getAge()
Inheritance	Child class inherits parent class	class Dog extends Animal
Polymorphism	Same method, different behaviour	method overloading/overriding
Abstraction	Hide implementation, show only interface	abstract class / interface

Inheritance

```
class Animal {
  void speak() { System.out.println("..."); }
}
class Dog extends Animal {
  void speak() { System.out.println("Woof"); }
}
```

Abstract Class vs Interface

- Abstract class: can have method body, one inheritance
- Interface: no method body (Java 7), multiple implementation
- Use abstract when sharing code; interface for contract

Overloading vs Overriding

- Overloading: same name, different parameters (compile-time)
- Overriding: child redefines parent method (runtime)

Access Modifiers

```
public    - anywhere
protected - same package + subclass
default   - same package only
private   - same class only
```

Constructor

- Same name as class, no return type
- Default constructor created if none defined
- Can be overloaded

Q: What is polymorphism?

A: Same method name behaves differently based on context. Compile-time: method overloading (same name, different params). Runtime: method overriding (child class redefines parent method). Example: Animal.speak() outputs different things for Dog and Cat.

Q: What is the difference between abstract class and interface?

A: Abstract class can have both abstract and concrete methods, constructor, and fields. Interface (Java 8+) can have default methods but is primarily a contract. A class can implement multiple interfaces but extend only one abstract class.

Q: What is encapsulation?

A: Wrapping data (fields) and methods together and restricting direct access using private. External access only through public getters and setters. Protects data integrity.

2. JAVA FUNDAMENTALS

Primitive Data Types

Collections Framework

```
int     - 32 bit integer
long    - 64 bit integer
double  - 64 bit decimal
float   - 32 bit decimal
boolean - true/false
char    - single character
byte    - 8 bit
short   - 16 bit
```

```
// ArrayList - dynamic array
ArrayList list = new ArrayList<>();
list.add(5);
list.get(0);
list.size();
list.remove(0);

// HashMap - key-value
HashMap map = new HashMap<>();
map.put("a", 1);
map.get("a");
map.containsKey("a");

// HashSet - unique values
HashSet set = new HashSet<>();
set.add(1);
set.contains(1);
```

String Methods

```
s.length()
s.charAt(0)
s.substring(1,3)
s.toUpperCase()
s.toLowerCase()
s.contains("abc")
s.replace("a","b")
s.split(",")
s.trim()
s.equals("abc")
```

Exception Handling

```
try {
  int x = 10/0;
} catch (ArithmeticException e) {
  System.out.println(e.getMessage());
} finally {
  // always runs
}
```

Loops

```
// for loop
for(int i=0; i<5; i++) {}

// while
while(condition) {}

// for-each
for(int x : list) {}

// do-while
do { } while(condition);
```

Checked vs Unchecked Exceptions

- Checked: must be handled (IOException, SQLException)
- Unchecked: runtime errors (NullPointerException, ArrayIndexOutOfBoundsException)

Static keyword

- static field: shared across all instances
- static method: called on class, not object
- static block: runs once when class loads

Q: What is the difference between == and .equals() in Java?

A: == checks reference equality (same object in memory). .equals() checks value equality (same content). For Strings always use .equals(). Example: new String('a') == new String('a') is false, but .equals() is true.

Q: What is a NullPointerException?

A: Thrown when you try to use a reference variable that points to null. Example: String s = null; s.length() throws NPE. Fix by checking null before use or using Optional.

Q: What is the difference between ArrayList and LinkedList?

A: ArrayList uses dynamic array — fast random access O(1), slow insert/delete in middle O(n). LinkedList uses doubly linked nodes — slow random access O(n), fast insert/delete O(1). Use ArrayList for most cases unless frequent insertions/deletions.

3. DATA STRUCTURES & ALGORITHMS

Structure	How it works	Time Complexity	Use case
Array	Fixed size, index-based	Access O(1), Search O(n)	Simple lists, known size
ArrayList	Dynamic array, resizes automatically	Access O(1), Insert O(n)	General purpose list
LinkedList	Nodes with next/prev pointers	Access O(n), Insert O(1)	Frequent insertions
Stack	LIFO — Last In First Out	Push/Pop O(1)	Undo, function calls

Queue	FIFO — First In First Out	Enqueue/Dequeue O(1)	Task scheduling
HashMap	Key-value pairs, hash function	Get/Put O(1) avg	Fast lookups by key
HashSet	Unique values, no duplicates	Add/Contains O(1) avg	Uniqueness checks
Binary Tree	Each node has max 2 children	Search O(log n) if balanced	Hierarchical data

Sorting Algorithms

Algorithm	How it works	Time	Space
Bubble Sort	Repeatedly swap adjacent if out of order	$O(n^2)$	O(1)
Selection Sort	Find min, put at start, repeat	$O(n^2)$	O(1)
Insertion Sort	Insert each element into correct position	$O(n^2)$	O(1)
Merge Sort	Divide into halves, sort, merge — divide & conquer	O(n log n)	O(n)
Quick Sort	Pick pivot, partition, recurse	O(n log n) avg	O(log n)
Binary Search	Search sorted array by halving search space	O(log n)	O(1)

Binary Search — Code

```
int binarySearch(int[] arr, int target) {
  int left = 0, right = arr.length - 1;
  while (left <= right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1; // not found
}
```

Big O — Time Complexity Quick Reference

Notation	Name	Example
O(1)	Constant	Array access, HashMap get
O(log n)	Logarithmic	Binary search, BST operations
O(n)	Linear	Loop through array, linear search
O(n log n)	Linearithmic	Merge sort, Quick sort
$O(n^2)$	Quadratic	Bubble sort, nested loops
O(2■)	Exponential	Recursive fibonacci

Q: What is the difference between Stack and Queue?

A: Stack is LIFO — Last In First Out. Like a pile of plates, last added is first removed. Queue is FIFO — First In First Out. Like a line of people, first in is first served. Stack uses push/pop, Queue uses enqueue/dequeue.

Q: What is recursion?

A: A function that calls itself until a base case is reached. Example: factorial(n) = n * factorial(n-1), base case factorial(0) = 1. Every recursive solution needs a base case to prevent infinite loop.

4. SQL — DATABASE QUERIES

Basic Queries

GROUP BY & HAVING

```sql
-- Select all
SELECT * FROM users;

-- Filter
SELECT * FROM users WHERE age > 18;

-- Sort
SELECT * FROM users ORDER BY name ASC;

-- Limit
SELECT * FROM users LIMIT 10;

-- Distinct
SELECT DISTINCT city FROM users;
```

```sql
-- Count users per city
SELECT city, COUNT(*)
FROM users
GROUP BY city;

-- Filter groups
SELECT city, COUNT(*)
FROM users
GROUP BY city
HAVING COUNT(*) > 5;
```

Aggregations

```sql
SELECT COUNT(*) FROM users;
SELECT AVG(salary) FROM employees;
SELECT MAX(age) FROM users;
SELECT MIN(age) FROM users;
SELECT SUM(amount) FROM orders;
```

JOINs

```sql
-- INNER JOIN
SELECT u.name, o.amount
FROM users u
INNER JOIN orders o ON u.id = o.user_id;

-- LEFT JOIN
SELECT u.name, o.amount
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

JOIN Type	Returns
INNER JOIN	Only rows with matching values in BOTH tables
LEFT JOIN	All rows from left table + matching from right (NULL if no match)
RIGHT JOIN	All rows from right table + matching from left (NULL if no match)
FULL JOIN	All rows from both tables (NULL where no match)

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. Example: WHERE age > 18 filters individual rows. HAVING COUNT(*) > 5 filters grouped results.

Q: What is a primary key vs foreign key?

A: Primary key uniquely identifies each row in a table — cannot be null, must be unique. Foreign key references the primary key of another table — creates relationship between tables. Example: orders.user_id is foreign key referencing users.id.

Q: What is an index in SQL?

A: A data structure that speeds up SELECT queries on a column. Like a book index — instead of scanning all rows, DB jumps directly to the result. Trade-off: faster reads, slower writes, more storage.

5. REST APIs

HTTP Methods

```
GET    - read/fetch data
POST   - create new resource
PUT    - update (full replace)
PATCH  - update (partial)
DELETE - remove resource
```

Status Codes

```
200 - OK
201 - Created
204 - No Content
400 - Bad Request
401 - Unauthorized
403 - Forbidden
404 - Not Found
422 - Validation Error
500 - Server Error
```

REST Principles

- Stateless — server stores no session state
- Client-Server — separated concerns
- Uniform Interface — consistent URL design
- Resource-based — URLs represent resources
- JSON — standard data format

URL Design Best Practices

```
GET    /users          - list all
GET    /users/1        - get one
POST   /users          - create
PUT    /users/1        - update
DELETE /users/1        - delete
```

Authentication

- JWT — JSON Web Token, stateless, sent in header
- OAuth2 — delegated auth, login with Google
- API Key — simple key passed in header/param

Q: What is REST?

A: Representational State Transfer. An architectural style for building APIs. Uses HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources identified by URLs. Stateless — each request contains all info needed.

Q: What is JWT?

A: JSON Web Token. A token-based authentication mechanism. Three parts: header.payload.signature encoded in Base64. Server generates token on login, client sends it in Authorization header on every request. Stateless — no session stored on server.

6. GIT — VERSION CONTROL

Command	What it does
git init	Initialize a new repository
git clone	Copy remote repo to local
git add .	Stage all changes
git commit -m 'msg'	Save staged changes with message
git push origin main	Upload commits to remote
git pull	Download and merge remote changes
git branch	Create new branch
git checkout	Switch to branch
git merge	Merge branch into current
git status	Show changed files
git log	Show commit history
git diff	Show unstaged changes
git stash	Temporarily save uncommitted changes
git reset HEAD~1	Undo last commit, keep changes

Q: What is the difference between git merge and git rebase?

A: Merge combines two branches and creates a merge commit — preserves full history. Rebase moves commits from one branch on top of another — creates cleaner linear history. Use merge for shared branches, rebase for local cleanup.

Q: What is a pull request?

A: A request to merge your branch into another branch (usually main). Code is reviewed by teammates before merging. Standard workflow: create branch, push changes, open PR, review, merge.

7. SPRING BOOT BASICS

Key Annotations

Basic REST Controller

```
@SpringBootApplication  // main class
@RestController         // REST API controller
@RequestMapping("/api") // base URL
@GetMapping("/users")   // GET endpoint
@PostMapping("/users")  // POST endpoint
@Autowired              // dependency injection
@Service                // business logic
@Repository             // data access layer
@Entity                 // JPA database entity
@Id                     // primary key
```

```
@RestController
@RequestMapping("/api")
public class UserController {

  @Autowired
  private UserService userService;

  @GetMapping("/users")
  public List getAll() {
    return userService.findAll();
  }

  @PostMapping("/users")
  public User create(@RequestBody User u) {
    return userService.save(u);
  }
}
```

Dependency Injection

- Spring creates and manages objects (beans)
- @Autowired injects dependencies automatically
- No need to use 'new' keyword for managed beans

Q: What is Spring Boot?

A: A Java framework that simplifies building production-ready applications. Provides auto-configuration, embedded server (no Tomcat setup), and dependency management via Maven/Gradle. Built on top of Spring Framework.

Q: What is dependency injection?

A: A design pattern where objects receive their dependencies from outside rather than creating them internally. Spring manages object creation and injects them via @Autowired. Makes code testable and loosely coupled.

8. COMMON CODING QUESTIONS — JAVA

Reverse a String

```
String reverse(String s) {
  StringBuilder sb = new StringBuilder(s);
  return sb.reverse().toString();
  // OR manually:
  // char[] c = s.toCharArray();
  // int l=0, r=c.length-1;
  // while(l  // return new String(c);
}
```

Check Palindrome

```
boolean isPalindrome(String s) {
  int l = 0, r = s.length()-1;
  while(l < r) {
    if(s.charAt(l) != s.charAt(r)) return false;
    l++; r--;
  }
  return true;
}
```

Find Duplicates in Array

```
void findDuplicates(int[] arr) {
  HashSet seen = new HashSet<>();
  for(int x : arr) {
    if(!seen.add(x)) System.out.println("Duplicate: " + x);
  }
}
```

Fibonacci

```
// Iterative (preferred)
int fib(int n) {
  if(n <= 1) return n;
  int a=0, b=1;
  for(int i=2; i<=n; i++) { int c=a+b; a=b; b=c; }
  return b;
}
```

```
// Recursive (simple but slow)
int fibRec(int n) {
  if(n <= 1) return n;
  return fibRec(n-1) + fibRec(n-2);
}
```

Two Sum Problem

```
// Find two numbers that add up to target
int[] twoSum(int[] nums, int target) {
  HashMap map = new HashMap<>();
  for(int i=0; i    int complement = target - nums[i];
    if(map.containsKey(complement))
      return new int[]{map.get(complement), i};
    map.put(nums[i], i);
  }
  return new int[]{};
}
```

9. iMOCHA TEST — TIPS & STRATEGY

Before starting:

- Use Chrome or Firefox — latest version
- Test camera and mic using system check link first
- Quiet room, good lighting, stable internet
- Close all other tabs and applications
- Don't click test link until fully ready — timer starts immediately

During the test:

- NEVER switch tabs or minimize browser — flagged immediately
- Stay in camera frame at all times
- Read ALL questions before answering MCQs
- For coding questions — write clean code with comments
- Submit each section before time runs out
- Don't leave any question blank — attempt all
- 91 minutes total — pace yourself, roughly 1-1.5 min per question

Expected sections:

- OOP concepts MCQs — 15-20 questions
- Java syntax and output prediction — 10-15 questions
- DSA — array/string problems — 2-3 coding questions
- SQL queries — 5-10 questions
- Logical/aptitude — 10-15 questions
- REST API / Git concepts — 5-10 questions

10. LAST MINUTE — KNOW THESE COLD

Q: What does JVM stand for?
A: Java Virtual Machine. Runs Java bytecode on any OS. Write once, run anywhere.

Q: What is garbage collection?

A: Automatic memory management in Java. JVM automatically frees memory of unused objects. No manual memory management needed unlike C++.

Q: What is the difference between int and Integer?

A: int is a primitive type. Integer is a wrapper class (object). Integer can be null, int cannot. Autoboxing converts between them automatically.

Q: What is a constructor?

A: Special method with same name as class, no return type. Called when object is created. Used to initialize fields.

Q: What is method overloading?

A: Multiple methods with same name but different parameters in same class. Resolved at compile time.

Q: What is the final keyword?

A: final variable: cannot be reassigned. final method: cannot be overridden. final class: cannot be inherited.

Q: What is an interface?

A: A contract that classes must implement. All methods are abstract by default (Java 7). A class can implement multiple interfaces.

Q: What is a HashMap collision?

A: When two keys produce same hash code. Java handles via chaining (linked list at that bucket) or open addressing.

Q: What is SOLID?

A: 5 OOP design principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

Q: What is the difference between throw and throws?

A: throw: actually throws an exception object inside method. throws: declares that method might throw an exception (in method signature).